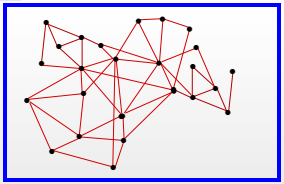
GRAPH EMBEDDINGS

- Graph embeddings; Laplacean Eigenmaps, Locally Linear Embeddings (LLE)
- Explicit mappings; PCA, LPP
- Graph factorization
- Graph Neural Networks, graph convolutional networks

Graph embedding

- *knn-graph construction* builds a graph to represent data [various divide-and-conquer techniques - not covered this year]
- *Graph embedding* does the opposite: maps a graph to data

Given: a graph that models some data (e.g., a kNN graph)



$\longrightarrow$ Data: $Y = [y_1, y_2, \cdots, y_n]$ in $\mathbb{R}^d$

- Trivial use: visualize a graph ($d = 2$)
- Wish: mapping should preserve similarities in graph.

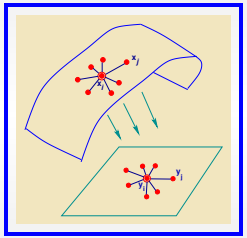
University of Minnesota

Vertex embedding: map every vertex x_i to a vector $y_i \in \mathbb{R}^d$

✦ Many applications [clustering, finding missing link, semi-supervised learning, community detection, ...]

- Graph captures similarities, closeness, ..., in data

Objective: Build a mapping of each vertex i to a data point $y_i \in \mathbb{R}^d$



- Many methods do this
- Eigenmaps and LLE are two of the best known

University of Minnesota

- Eigenmaps uses the *graph Laplacean*
- Recall: Graph Laplacean is a matrix defined by : $\boxed{L = D - W}$

$$\begin{cases} w_{ij} \geq 0 & \text{if } j \in Adj(i) \\ w_{ij} = 0 & \text{else} \end{cases} \qquad D = \text{diag}\left[d_{ii} = \sum_{j \neq i} w_{ij} \right]$$

with $Adj(i)$ = neighborhood of i (excludes i)

- Remember: vertex i represents data item x_i. W'll use i or x_i to refer to the vertex.
- We will find the y_i's by solving an optimization problem.

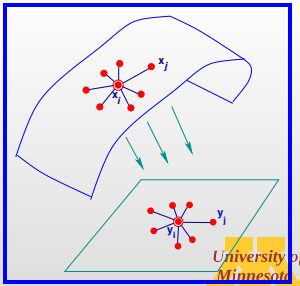
The Laplacean eigenmaps approach

Laplacean Eigenmaps [Belkin-Niyogi '01] *minimizes*

$$\mathcal{F}(Y) = \sum_{i,j=1}^{n} w_{ij} \|y_i - y_j\|^2 \quad \text{subject to} \quad YDY^\top = I$$

Motivation: if $\|x_i - x_j\|$ is small (orig. data), we want $\|y_i - y_j\|$ to be also small (low-Dim. data)

- Original data used indirectly through its graph
- Objective function can be translated to a trace (see Property 3 in Lecture notes 9) and will yield a sparse eigenvalue problem



- Problem translates to:
$$\begin{cases} \min & \text{Tr}\left[Y(D-W)Y^\top\right] . \\ Y \in \mathbb{R}^{d\times n} \\ YD\,Y^\top = I \end{cases}$$

- Solution (sort eigenvalues increasingly):
$$(D-W)u_i = \lambda_i D u_i\ ; \quad y_i = u_i^\top ; \quad i = 1, \cdots, d$$

- An $n \times n$ sparse eigenvalue problem [In 'sample' space]

- Note: can assume $D = I$. Amounts to rescaling data. Problem becomes
$$(I-W)u_i = \lambda_i u_i\ ; \quad y_i = u_i^\top ; \quad i = 1, \cdots, d$$

Locally Linear Embedding (Roweis-Saul-00)

- LLE is very similar to Eigenmaps. Main differences:

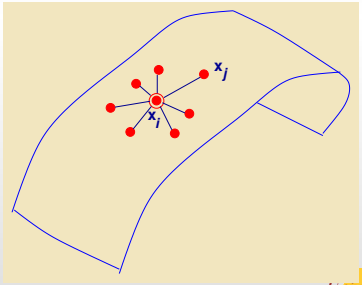
1) Graph Laplacean matrix is replaced by an 'affinity' graph

2) Objective function is changed: want to preserve graph

1. Graph: Each x_i is written as a convex combination of its k nearest neighbors:

$$x_i \approx \Sigma w_{ij} x_j, \quad \sum_{j \in N_i} w_{ij} = 1$$

- Optimal weights computed ('local calculation') by minimizing $\|x_i - \Sigma w_{ij} x_j\|$ for $i = 1, \cdots, n$



University of Minnesota

2. Mapping: The y_i's should obey the same 'affinity' as x_i's $\rightsquigarrow$

Mininize $\sum_i \left\| y_i - \sum_j w_{ij} y_j \right\|^2$ subject to: $Y\mathbb{1} = 0, \quad YY^\top = I$

Solution: $(I - W^\top)(I - W)u_i = \lambda_i u_i; \qquad y_i = u_i^\top$.

- $(I - W^\top)(I - W)$ replaces the graph Laplacean of eigenmaps

Implicit vs explicit mappings

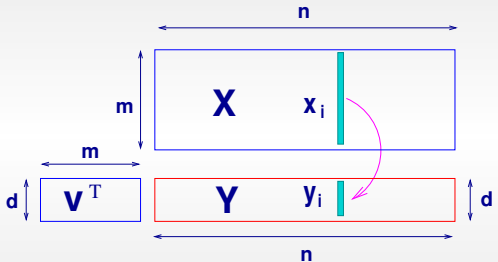
- In Eigenmaps and LLE: determine a set of $y_i's$ in $\mathbb{R}^d$ from data points $\{x_i\}$.
  - The mapping $y_i = \phi(x_i), i = 1, \cdots, n$ is implicit
- Difficult to compute a y for an x that is not one of the x_i's
- Inconvenient for classification. Thus is known as the "The out-of-sample extension" problem
- In Explicit (also known as linear) methods: mapping ϕ is known explicitly (and it is linear.)

Locally Preserving Projections (He-Niyogi-03)

- LPP is a linear dimensionality reduction technique

- Recall the setting:

Want $V \in \mathbb{R}^{m \times d}$; $Y = V^\top X$



- Starts with the same neighborhood graph as Eigenmaps: $L \equiv D - W$ = graph 'Laplacean'; with $D \equiv diag(\{\Sigma_i w_{ij}\})$.

- Optimization problem is to solve

$$\min_{Y \in \mathbb{R}^{d \times n},\ YDY^\top = I} \quad \Sigma_{i,j} w_{ij} \|y_i - y_j\|^2, \quad Y = V^\top X.$$

- Difference with eigenmaps: Y is an explicit projection of X
- Solution (sort eigenvalues increasingly) $XLX^\top v_i = \lambda_i XDX^\top v_i \quad y_{i,:} = v_i^\top X$
- Note: essentially same method in [Koren-Carmel'04] called 'weighted PCA' [viewed from the angle of improving PCA]

More recent methods

- node2vec, DeepWalk, GraRep, ... See the following papers ... among many others :

[1] *William L. Hamilton, Rex Ying, and Jure Leskovec* Representation Learning on Graphs: Methods and Applications arXiv:1709.05584v3

[2] *Shaosheng Cao, Wei Lu, and Qiongkai Xu* GraRep: Learning Graph Representations with Global Structural Information, CIKM, ACM Conference on Information and Knowledge Management, 24

[3] *Amr Ahmed, Nino Shervashidze, and Shravan Narayanamurthy*, Distributed Large-scale Natural Graph Factorization [Proc. WWW 2013, May 13–17, 2013, Rio de Janeiro, Brazil]

University of Minnesota

Example: Graph factorization

- Line of work in Papers [1] and [3] above + others
- Instead of minimizing $\sum w_{ij} \|y_i - y_j\|_2^2$ as before

  ... try to minimize $\sum_{ij} |w_{ij} - y_i^T y_j|^2$
- In other words solve: $\min_Y \|W - Y^T Y\|_F^2$
- Referred to as *Graph factorization*
- Common in knowledge graphs

From Graph Embeddings to Graph Neural Networks

- We have seen how to embed nodes using the graph structure.
- A Graph Convolutional Network (GCN) integrates node features and uses a neural network to build embedding
- Input: node features $\boldsymbol{X} \in \mathbb{R}^{n \times d}$ and graph adjacency $\boldsymbol{A}$.
- Output: node representations $\boldsymbol{X}^{(\ell)}$ updated layer by layer.
- Main idea: each node repeatedly averages information from its neighbors.

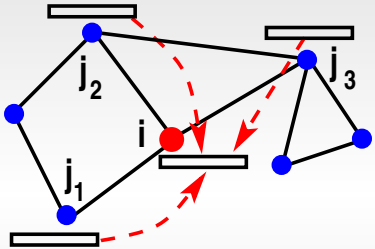
✦ Next table: differences between classical embeddings and GCN [Thanks: AI]

University of Minnesota

Feature	Classical Graph Embeddings (e.g., LLE, Laplacian Eigenmaps)	Graph Convolutional Networks (GCN)
Learning Paradigm	Matrix factorization / Dimensionality reduction	Deep Learning / Neighborhood Aggregation
Node Features	Ignores external node features (uses structure only)	Integrates node features (text, images, attributes)
Inductive Capability	**Transductive**: Cannot easily embed new, unseen nodes	**Inductive**: Can embed new nodes/graphs without retraining
Scalability	Poor for large graphs (requires expensive matrix math)	Highly scalable (uses mini-batches and SGD)
Parameters	Direct embedding lookup (parameter size grows with nodes)	Shared weight matrices (fixed parameters regardless of graph size)

Message Passing

- A feature at a given node averages features from nearest neighbors. Referred to as message passing.
- Node features are rows of a feature matrix $X^{(l)}$ at layer l
- Message passing: $X^{(l)} \rightarrow \hat{A}X^{(l)}$ What is $\hat{A}$?



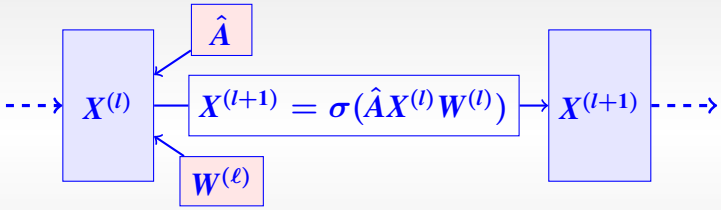
$$\hat{A} = \tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2} \text{ with } \tilde{A} = A + I, \tilde{D} = \text{Diag}(\tilde{A})$$

GCN Layer: Basic Form

- Add self-loops: $\widetilde{A} = A + I$.
- Let $\widetilde{D}$ be the degree matrix of $\widetilde{A}$. Define $\boxed{\widehat{A} = \widetilde{D}^{-1/2}\widetilde{A}\widetilde{D}^{-1/2}}$
- A common GCN layer is $\boxed{X^{(\ell+1)} = \sigma\left(\widehat{A}\, X^{(\ell)} W^{(\ell)}\right).}$
- Here $X^{(0)} = X$, and $W^{(\ell)}$ is learned.

University of Minnesota

Structure of l-th layer



The l-th layer of Graph Convolutional Network (GCN)

Interpretation

- The matrix $\boxed{\widehat{\boldsymbol{A}} = \widetilde{\boldsymbol{D}}^{-1/2}\widetilde{\boldsymbol{A}}\widetilde{\boldsymbol{D}}^{-1/2}}$ performs normalized neighbor averaging.
- The multiplication by $\boldsymbol{W}^{(\ell)}$ learns new feature combinations.
- The nonlinearity σ gives the model expressive power.
- After $\boldsymbol{k}$ layers, a node has received information from nodes up to $\boldsymbol{k}$ hops away.
- Thus GCNs combine graph smoothing with trainable feature transformation.

University of Minnesota

- For semi-supervised node classification, only some node labels are known.
- A simple two-layer GCN is $$\boldsymbol{Z} = \mathbf{softmax}\left(\widehat{\boldsymbol{A}}\,\boldsymbol{\sigma}(\widehat{\boldsymbol{A}}\boldsymbol{X}\boldsymbol{W}^{(0)})\boldsymbol{W}^{(1)}\right),$$
- The loss is computed only on labeled nodes.
- The graph propagates information from labeled to nearby unlabeled nodes.

University of Minnesota

Example of Application: GCN for Node Classification

What we are Given:

- *The Graph Structure:* matrix A == adjacency matrix of graph
- *Node Features:* Attributes or properties for each node (e.g., text, numerical data, or user profiles).
- *Partial Labels:* True labels for a subset of the nodes (the training set).

What we must find: Missing Labels for the unlabeled nodes (the test set)

University of Minnesota

GCN for Node Classification

- Common approach: Last layer is a feature vector of dimension C, number of classes. Result: a row vector output $z = [z_1, \cdots, z_C]$
- Apply *softmax* to obtain probabilities: $\hat{y}_{i,c} = \dfrac{\exp(z_{i,c})}{\sum_{j=1}^{c} \exp(z_{j,c})} i = 1, \cdots, n$
- Cross Entropy Loss (here $y_{i,c}$ = true label): $\mathcal{L}_i = -\sum_{c=1}^{C} y_{i,c} \log \hat{y}_{i,c}$
- Training: Embedding (all nodes) - followed by loss (labeled nodes only) [SEE code sample.

University of Minnesota

Example of application: graph classification

- Input: graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ - $\mathcal{V}$ = vertex set, $\mathcal{E}$ = edge set
- Represent graph by its adjacency matrix A (sparse) = $a_{ij} \in \{0, 1\}$
- In graph classification the input $\equiv$ a set of graphs $\{\mathcal{G}_1, \cdots, \mathcal{G}_m\}$.
- So m adjacency graphs $A_1, \cdots, A_m$ and m feature matrices $X_1, X_2, \cdots, X_m$.

If $|\mathcal{V}_k| = n_k$ then size of A_k= $n_k \times n_k$ and size of X_k = $n_k \times d_k$.

- Assume at first that there is no mini-batching performed. Then for all layers except the last one, GCN works

Graph classification

$$\hat{A} = \begin{pmatrix} \hat{A}_1 & & & \\ & \hat{A}_2 & & \\ & & \ddots & \\ & & & \hat{A}_m \end{pmatrix} \in \mathbb{R}^{n \times n}, \qquad X^{(\ell)} = \begin{pmatrix} X_1 \\ X_2 \\ \vdots \\ X_m \end{pmatrix} \in \mathbb{R}^{n \times d_\ell}.$$

When $l = L$ in the matrix $X^{(L+1)}$ obtained at the last layer will be $n \times d_{L+1}$.

- Need to embed each graph $\mathcal{G}_k$ into a (row) vector $z_{\mathcal{G}_k}$ of length C: the c-th component holds a probability that $\mathcal{G}_k$ is in class c.
- The dimension d_{L+1} should therefore be selected to be equal to C.
- To classify graphs, the matrix $X^{(L+1)}$ of last layer will be $n \times d_{L+1}$ with $d_{L+1} = C$.
- In addition, at the last layer GCN applies a 'global pooling' operation
- All the node features for the same graph $\mathcal{G}_k$ are averaged $\rightarrow$ a single representation (a row of length C) for the graph

University of Minnesota

Pooling: one of several methods

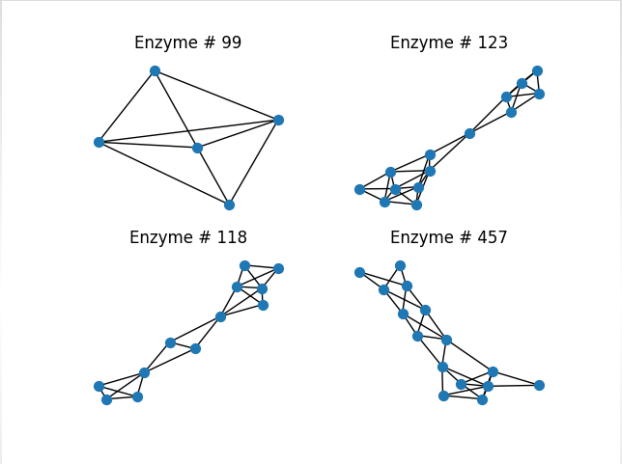
- Apply a small MLP to create a C-dimensional graph encoding, which is a vector representation of the graph computed as

$$z_{\mathcal{G}_k} = \texttt{MLP}\left(\underset{i \in \mathcal{V}_k}{\texttt{mean}}\, x_i^{(L+1)}\right) \in \mathbb{R}^C \quad \text{where} \quad \underset{i \in \mathcal{V}_k}{\texttt{mean}}\, x_i^{(L+1)} = \frac{1}{|\mathcal{V}_k|} \sum_{i \in \mathcal{V}_k} x_i^{(L+1)}$$

- Using the full set of all graphs is not practical. Instead, it is common to resort to mini-batching whereby a set of ν graphs is selected at random at each step.

University of Minnesota

Illustration: ENZYME data set dataset: 600 molecules represented by graphs that translate their protein structure. An illustration of 4 such graphs is shown



- Six classes corresponding to six main categories of enzymes: Oxidoreductases, Transferases, Hydrolases, Lyases, Isomerases, and Ligases.

ENZYME example (cont.)

Problem: given a molecule with a given structure (graph) and the nature of the compounds of the nodes in the graph (features) determine what type of enzyme it is

- It turns out that each node has a feature vector of length 3.
- Sizes of the 4 graphs: $n_1 = 5, n_2 = 14, n_3 = 12$, and $n_4 = 15$.
- If this set were a certain batch then for this batch then: $X^{(0)} \in \mathbb{R}^{46\times 3}$, $X^{(l)} \in \mathbb{R}^{46\times d_l}$, & $X^{(L+1)} \in \mathbb{R}^{46\times d_{L+1}}$,
- Setting $d_{L+1} = 6$ classes, + pooling $\longrightarrow Z \in \mathbb{R}^{4\times 6} \longrightarrow$ 4 graphs and 6 classes.

University of Minnesota

GraphSAGE

- *GraphSAGE* (Graph Sample and Aggregation) == a scalable and efficient graph neural network architecture designed to handle large-scale graphs
- Feature: uses sampling-based approach to aggregate information from a fixed-size neighborhood

In *GraphSAGE*, the feature representation $x_i^{(\ell)}$ of each node v_i is updated by aggregating information from its ℓ-hop neighbors. Update rule at ℓ-th layer:

$$x_i^{(\ell+1)} = \texttt{ReLU}\Big(W^{(\ell),T} \times \texttt{AGGREGATE}\left(\{x_j^{(\ell)} : j \in \mathcal{N}(v_i)\} \cup \{x_i^{(\ell)}\}\right)\Big),$$

- `AGGREGATE` combines features of neighboring nodes.
- Common aggregation functions include mean, LSTM, and pooling.

University of Minnesota

GraphSAGE uses a sampling technique to limit the size of the neighborhood: For each node, a fixed number of neighbors are randomly sampled to aggregate features, i.e., $\mathcal{N}(v_i) = \texttt{sample}(v_i)$.

- Simplest mean-aggregation function, computes the element-wise mean of the neighboring node features:

$$\texttt{AGGREGATE}\left(\{x_j : j \in \mathcal{N}(v_i)\}\right) = \frac{1}{|\mathcal{N}(v_i)|} \sum_{j \in \mathcal{N}(v_i)} x_j.$$

GraphSAGE Embedding

Input: Graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$, input features $\{x_v, \forall v \in \mathcal{V}\}$, neighborhood function $\mathcal{N}$

Parameters: depth K, weight matrices $W^{(\ell)}, \forall \ell \in \{1, \cdots, K\}$

Output: $z_v, \forall v \in \mathcal{V}$ ▷ vector representations after message passing completion

for $\ell = 1, \cdots, K$ **do**
  for $v \in \mathcal{V}$ **do**
    $x^{(\ell)}_{\mathcal{N}(v)} \leftarrow \texttt{AGGREGATE}(\{x^{(\ell-1)}_u, \forall u \in \mathcal{N}(v)\})$
    $x^{(\ell)}_v \leftarrow \texttt{ReLU}\left(W^{(\ell),T}\left[x^{(\ell-1)}_v \parallel x^{(\ell)}_{\mathcal{N}(v)}\right]\right)$ ▷ $\parallel$ denotes concatenation
  end for
  $x^{(\ell)}_v \leftarrow x^{(\ell)}_v / \|x^{(\ell)}_v\|_2, \quad \forall v \in \mathcal{V}$ ▷ normalization
end forreturn $z_v \leftarrow x^{(K)}_v, \quad \forall v \in \mathcal{V}$